Databases for Big Data 2025/2026
Lecture Notes for Week 6
Dmytro Tupkalenko (89252101)

Lecture Title: Query Scheduling & Coordination

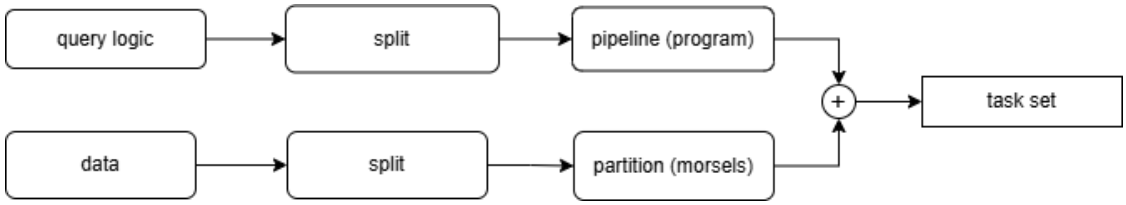
1 Query Execution Overview

Query execution includes a Directed Acyclic Graph (DAG) of operators.

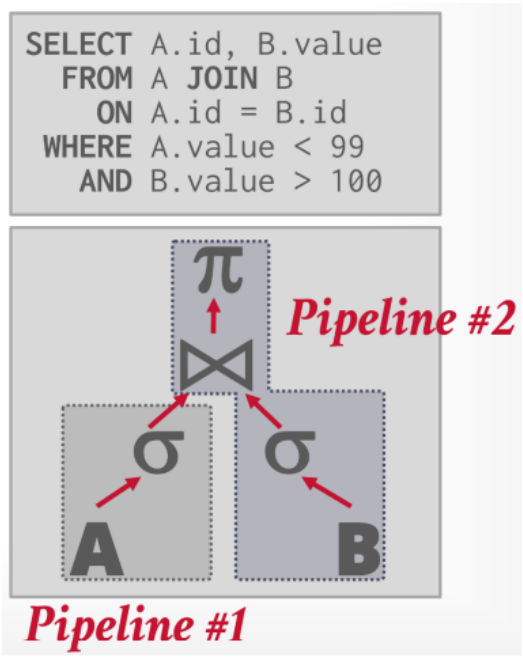
- An **operator instance** represents an operator on a unique segment of data.
- A **task** is a sequence of one or more operator instances.
- A **task set** is the collection of executable tasks for a logical pipeline.

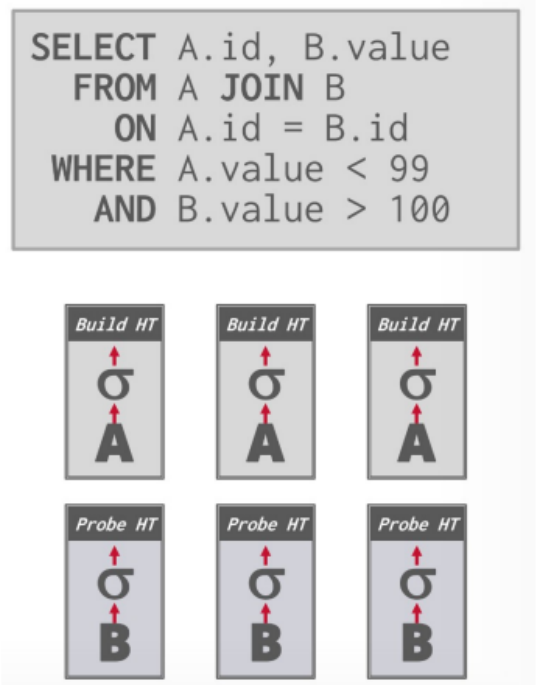
The execution of a query involves splitting it into pipelines with clearly determined dependencies, which define the order of execution. The data itself is divided into packages, and processes are carried out for each package individually, with tables typically partitioned in some manner, such as by morsels.

Diagram:



Example: Image to the left contains splitting of pipelines (note the ordering, the first pipeline corresponds to the build phase of Hash join, whereas the second to the probe phase - which we cannot execute in a different order. Image to the right presents formed tasks set





2 Scheduling in DBMS

For each query plan, the DBMS must decide where, when, and how to execute the tasks. These decisions should take into account the hardware and the statistical distribution of the data, as well as factors such as task allocation, which core to use, and where the task should store its output.

Goals (what should the DBMS optimize for):

- 1: No query should be left unexecuted.
- 2: Execution should be adapted to skewness of the data.
- 3: Most of the time should be spent on executing the queries, not their management.

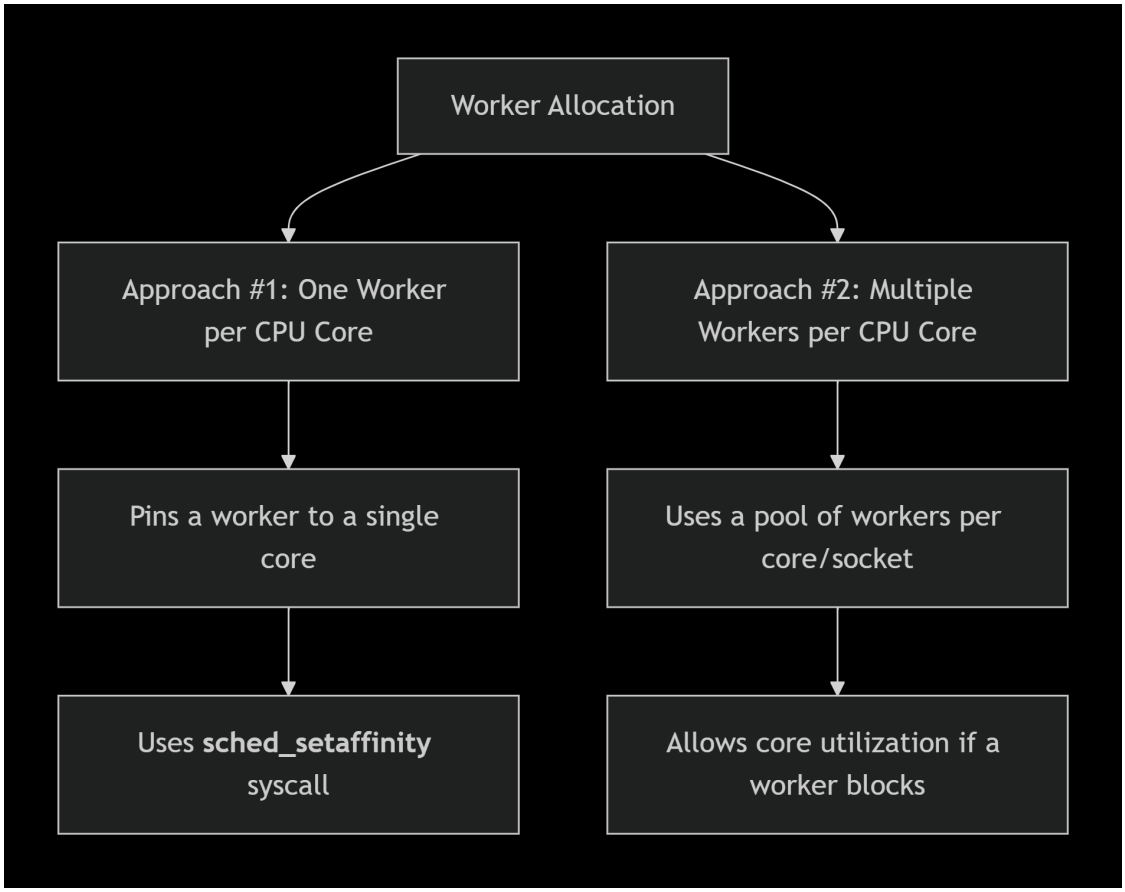
3 Process Model & Workers

Process model - defines how the system supports the concurrency of requests.
Worker - part of DBMS that is responsible for execution of the tasks.

4 Worker Allocation

For worker allocation there are two approaches. Approach #1 (One Worker per CPU Core), Approach #2 (Multiple Workers per CPU Core).

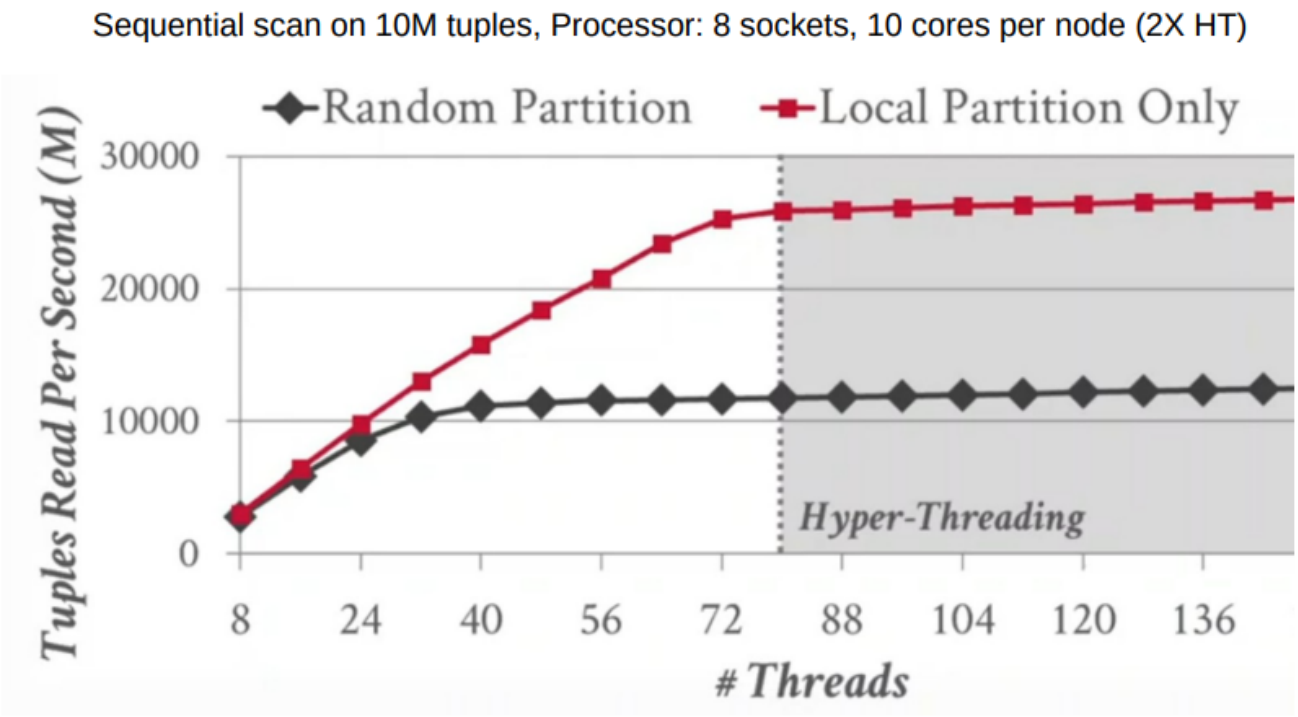
Diagram:



5 Data Placement & Memory Access

Key Principle: In NUMA systems, local data placement dramatically outperforms random or remote placement.

Why? Access to remote memory is expensive - even accessing the memory through interconnect (which is fast) is more expensive than accessing the local memory.



Note: HyperThreading does not help here. **Why?** Because a CAS (compare-and-swap) operation executed by multiple threads allows only one to succeed. This creates contention, since all threads must access the same memory location - leading to what is known as a CAS storm.

The cache-coherency lines connecting the cores must stay synchronized, and when many CAS operations target the same location, this synchronization causes intensive ping-ponging between cores, significantly degrading performance.

6 Partitioning vs. Placement

Data distribution involves two key decisions: how to split the data (Partitioning) and which physical resources hold the splits (Placement):

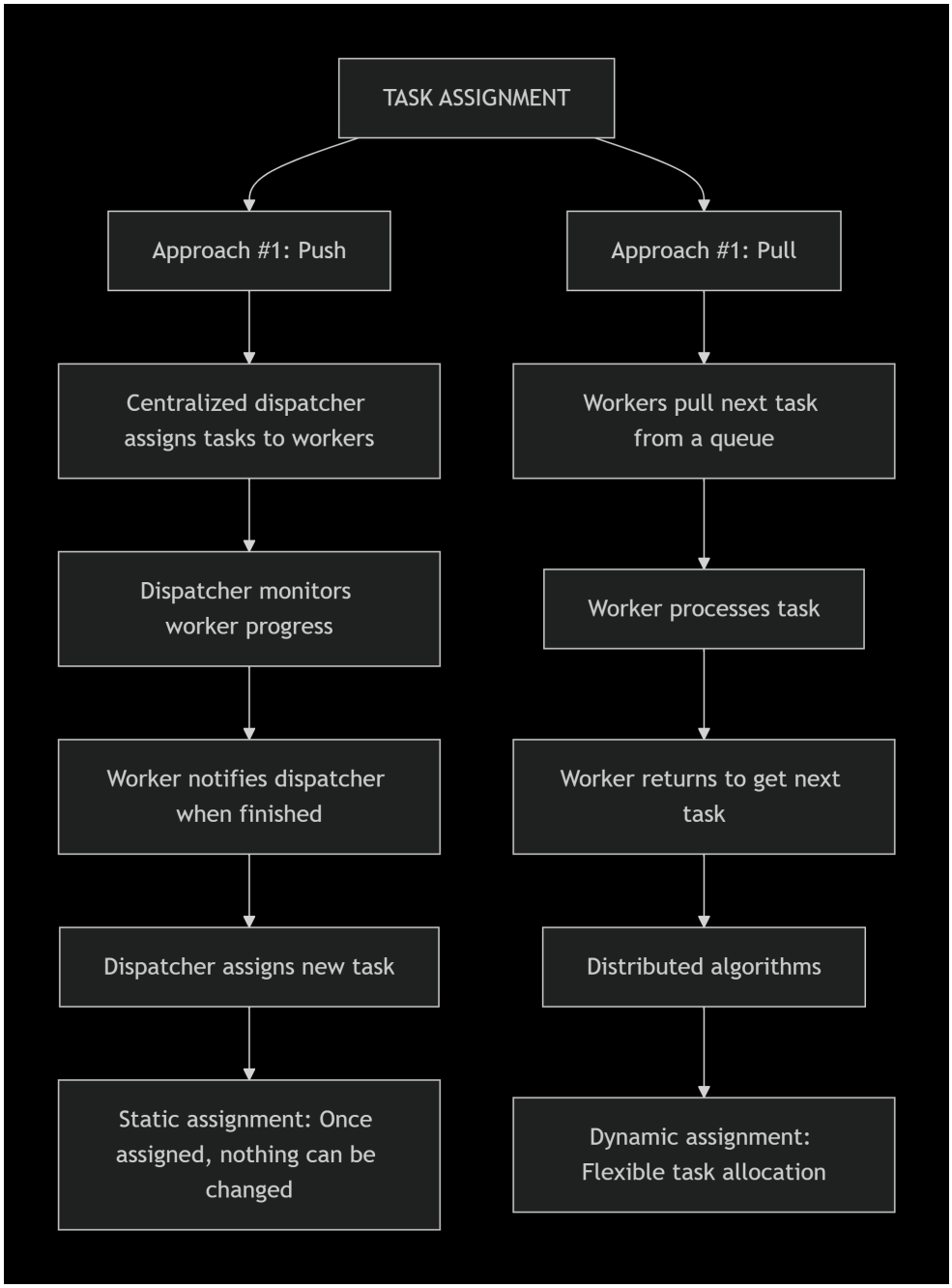
- **Partitioning Scheme:** (How to split the dataset)
  - Round Robin (split the data in equal parts).
  - Attribute Range (split the data by the range of some attribute - speed up of some processes).
  - Hashing (apply a hash function to one or more attributes of each record and assign the record to a partition based on the hash value.
  - Replication (the same data is stored on multiple locations)
- **Placement Scheme:** (Where to put partitions)
  - Round-Robin
  - Interleaved across workers. (Round-Robin + shift)

7 Task Assignment

Task assignment process defines how pending tasks are handed to worker threads.

There are two approaches. Approach #1 (Push) - dispatcher distributes the tasks across workers, Approach #2 (Pull) - workers pull for the next task from the pool of available tasks.

Diagram:



Observation: Regardless of the worker allocation or task assignment policy used, it is crucial that workers operate on local data. The DBMS scheduler must be aware of both the data locations and the memory layout of each node.

8 Scheduling Strategies

Core question: Given a logical query plan, how does the DBMS decide which physical tasks (operators) to create and when to schedule them? **Multiple approaches:**

8.1 Static Scheduling

- Degree of parallelism (DOP) is decided once, during query compilation or at start of execution.
- Fixed pipeline structure and fixed number of worker threads for the entire query lifetime.
- Simple, predictable, but cannot react to changing system load or data skew.

8.2 Morsel-Driven Scheduling

- Table data is partitioned into small, fixed-size **morsels**.
- Morsels are completely independent and can be processed safely on any core.
- Exactly one worker thread per physical core, pinned with CPU affinity.
- Workers *pull* the next available morsel from a shared or partitioned queue (pull model).
- Morsels are distributed round-robin or via NUMA-aware routing to guarantee roughly equal work and strong data locality.

8.3 Stride Scheduling

- Classic proportional-share algorithm (originally from OS research).
- Each query is assigned tickets (weight). Stride = constant / tickets (lower stride = higher priority).
- Each time a query is scheduled, its *pass* value is increased by its stride.
- The scheduler always picks the query with the current lowest pass value.
- Guarantees CPU time is allocated proportional to assigned tickets over the long term.

Example: Three concurrent queries A (medium priority), B (low priority), C (high priority) with strides 100, 200, 40 (pass starts at 0). The scheduler repeatedly selects the query with the smallest pass, producing the fair sequence shown below.

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

9 Key Components of Dynamic Scheduling System to Remember

- **Slot**: Worker thread.
- **Morsel**: Small, independent unit of work + pipeline code.
- **Task Set**: Group of morsels for one task.
- **Global Queue**: Holds all ready-to-execute morsels.
- **Local Queue**: Queue of task sets, used to generate new morsels as dependencies finish.
- **Scheduler**: Picks morsels from global queue based on priority.

10 HyPer Architecture: Morsel-Driven Scheduling

HyPer introduced the **morsel-driven** parallel execution model (the mandatory paper this week describes this system in more details):

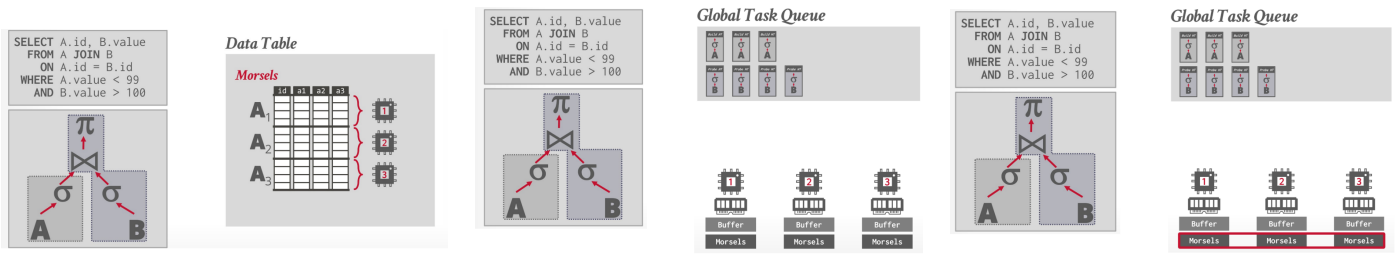
Core Concepts

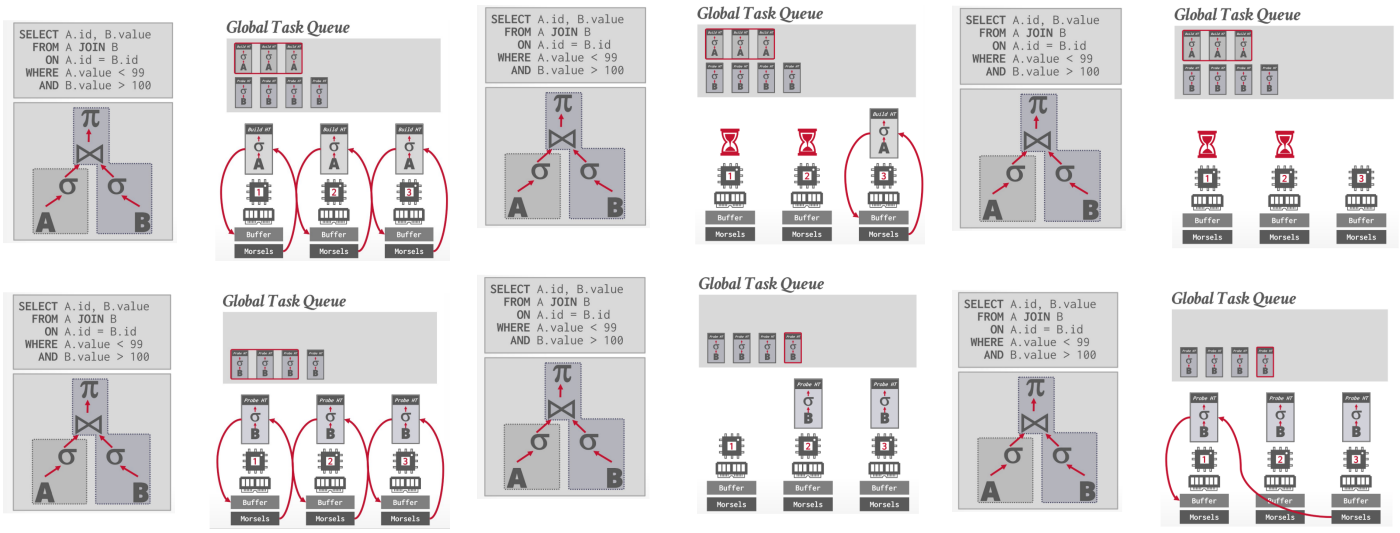
- **Data Partition:** Break down a query's execution into morsels.
- **No Separate Dispatcher:** No dedicated dispatcher thread (that is no thread is occupied by management of morsel distribution across threads).
- **Cooperative Scheduling:** Worker threads cooperatively pull tasks from a **single global task queue.**
- **Locality-Aware:** Workers try to select tasks that will execute on data local to their NUMA node.
- **Work Stealing Mechanism**
  - **Problem:** With one worker per core and one morsel per task, workers could sit idle if they finish while others are busy.
  - **Solution:** Implement **work stealing** where idle workers "steal" tasks from busy workers.

Limitations of HyPer Approach

- **Uniform Task Cost Assumption:** Treats every task as having same execution cost (unrealistic).
- **No Execution Priorities:** All query tasks execute with same priority.

Example of Query Execution





11 Umbra Architecture: Morsel Scheduling 2.0

Core Concepts

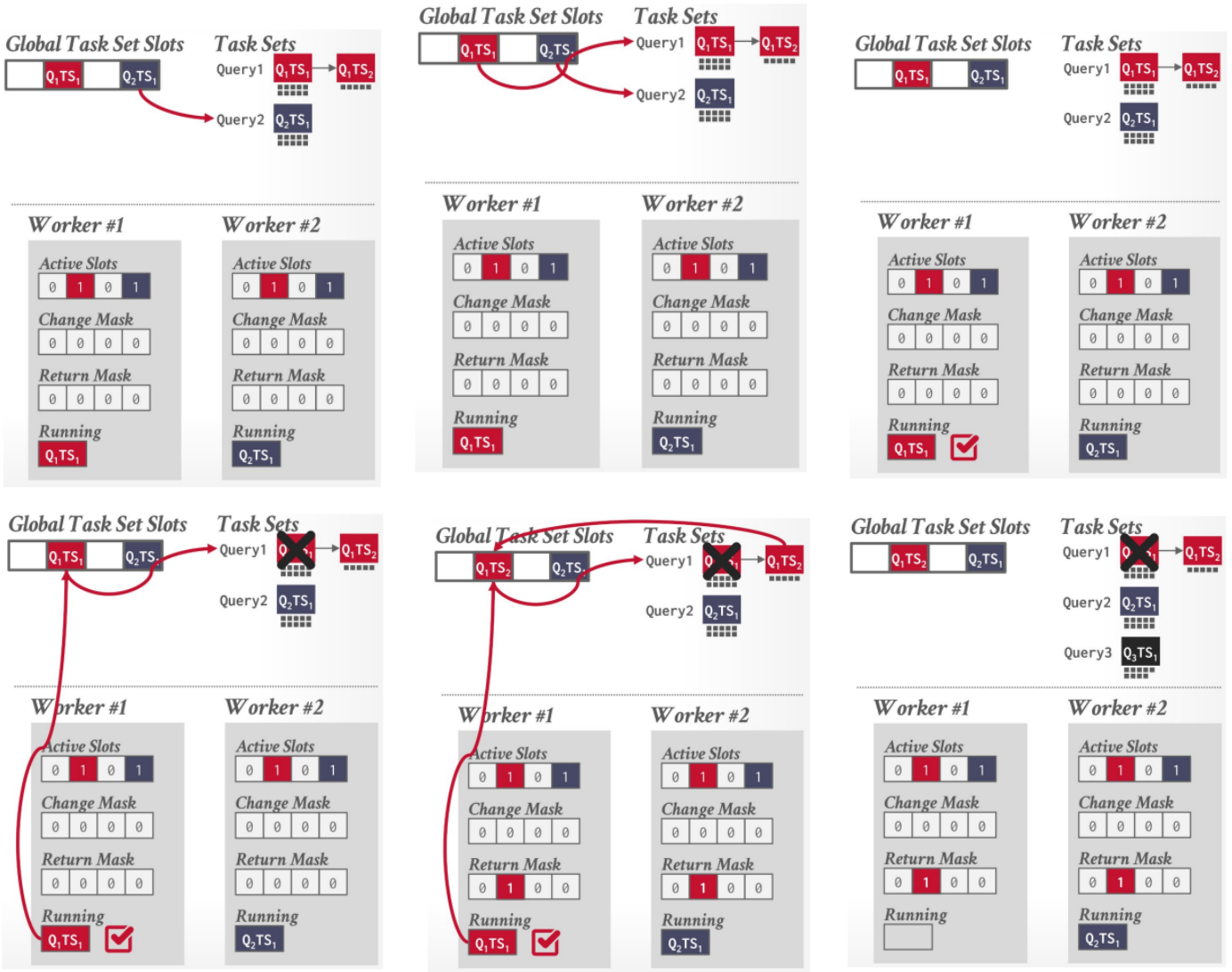
- Implements **modern stride scheduling** variant.
- Tasks not created statically <-> System starts with small morsels and **exponentially increases their size** until individual task takes ∼1ms to execute.
- Ensures **short-running queries finish quickly** while **long-running queries are not starved**.
- Single **global queue of morsels** and **local queue of task sets per query**.
- Scheduling is **query-aware**: picks next morsel from query with **lowest "pass value"** (highest priority).
- Each worker maintains **Thread-Local Storage (TLS)** with low-latency.
- Each worker maintains its own thread-local meta-data about the available tasks to execute:
  - **Active Slots:** Keeps track of which entries in the global slot array currently contain active task sets. This allows the worker to quickly identify tasks that are ready for execution.
  - **Change Mask:** Signals when a new task set has been added to the global slot array. This helps workers stay updated without constantly scanning the entire array.
  - **Return Mask:** Signals when a worker has completed a task set. Other workers can use this information to avoid idle waiting and efficiently pick up new tasks.
  - Workers use CAS operations to update meta-data to broadcast the changes.

Query Execution Flow

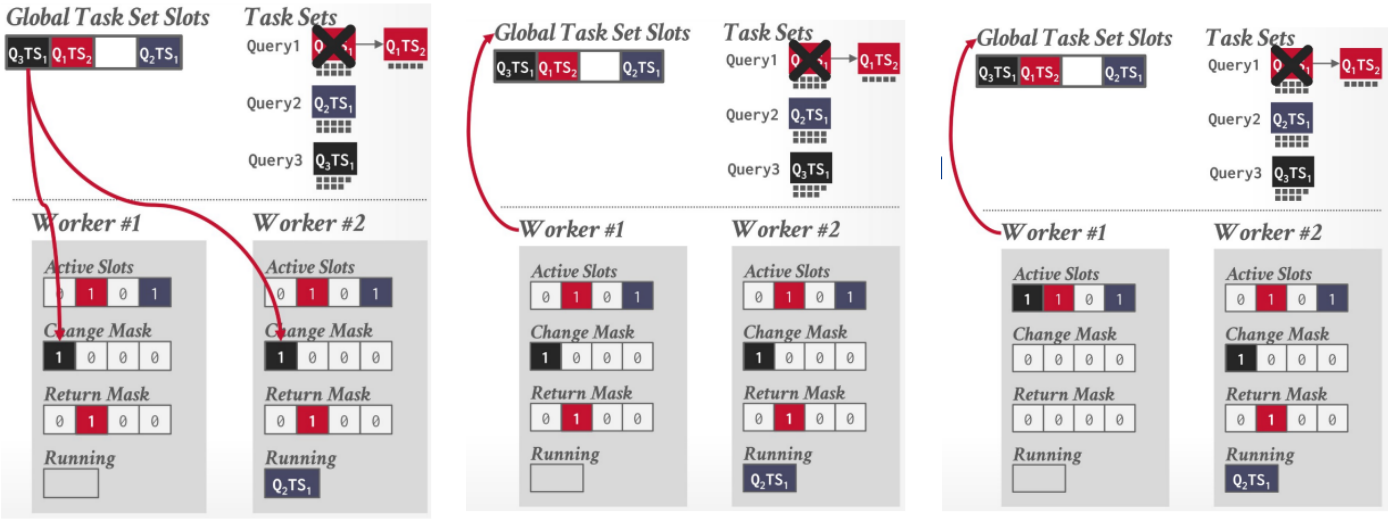
1. **Query Submission:**
   - Query arrives, first ready task set(s) created
   - Scheduler enqueues morsels into **global queue**
2. **Morsel Scheduling:**
   - Each **Slot** repeatedly fetches morsel from **global queue**
   - **Scheduler** ensures fairness by selecting from query with **lowest pass value**

3. **Task-Set Progression:**
    - Slots complete morsels, mark completion bit in task set's **return mask**
    - When **all bits set** (all morsels done), task set complete
    - Triggers activation of **dependent task sets**, their morsels enqueued
4. **Parallelism and Fairness:**
    - Multiple slots can work on morsels from same task set concurrently
    - Idle slots pick morsels from any query, respecting query-level priority
5. **Completion:**
    - When query's final task set finishes, query complete and resources freed

Example of Scheduling State



NEXT PAGE ->



12 Final Thoughts

Do not rely on the OS to manage DBMS query execution.

DBMSs are specifically optimized for data-management workloads and architectures. As a result, they generally outperform operating systems, which are designed for more general-purpose tasks.

13 Tools Used

- Overleaf for generating the report.
- Draw.io for drawing the diagrams (the one with the white background).
- deepseek for generating the other diagrams (the ones with the black background)
- deepseek and ChatGPT for refinement of the notes I have taken in class.